

The xv6 Kernel Event Recorder

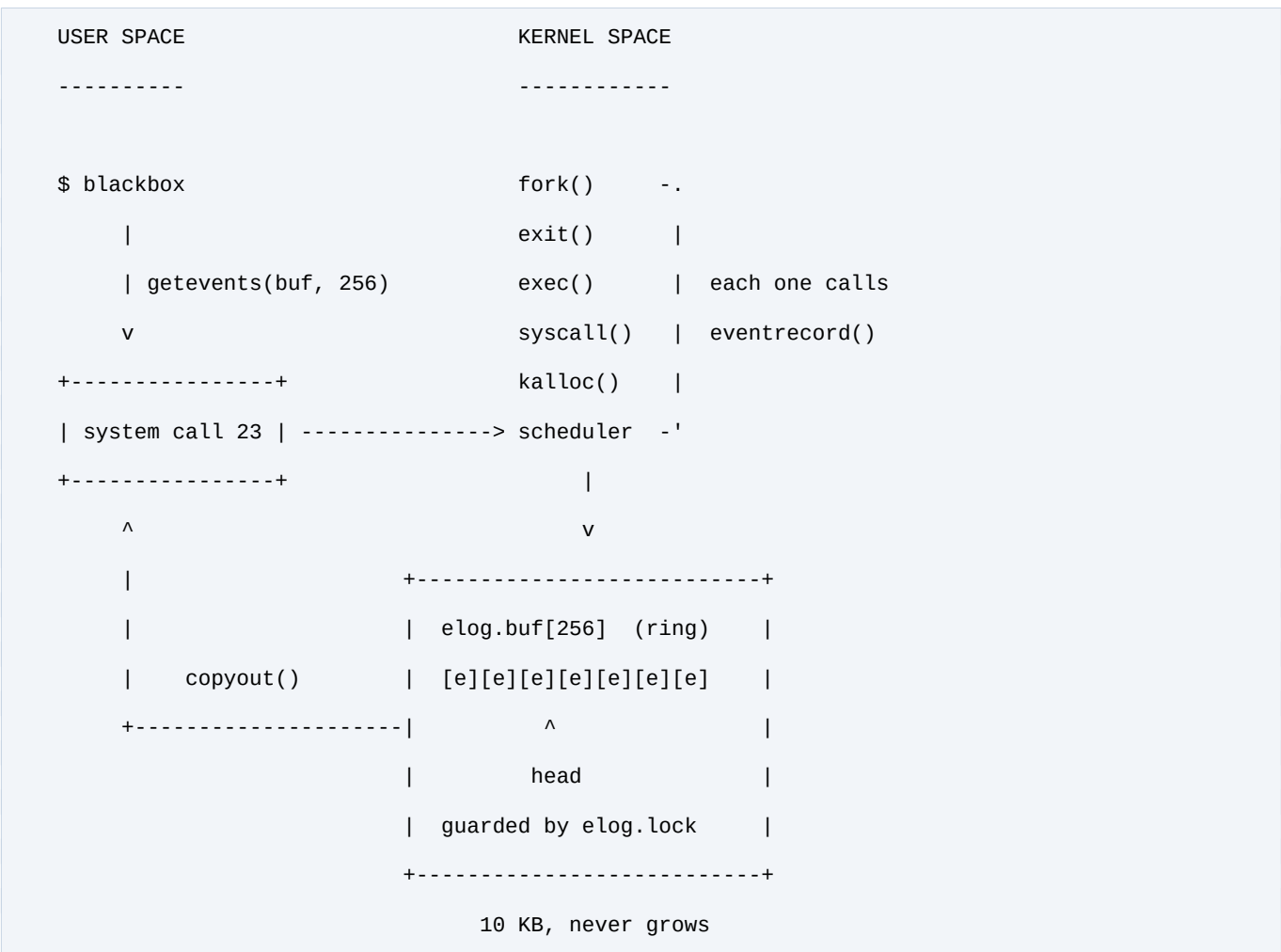
A "black box" flight recorder for the kernel — a step-by-step guide

1. What it is, in one paragraph

An aeroplane carries a black box that continuously records what the aircraft is doing. It holds only the last few hours: new data writes over the oldest. If something goes wrong, investigators read the box to see what happened just before.

This project puts the same idea inside the xv6 kernel. Whenever something interesting happens — a process forks, exits, runs a program, makes a system call, gets scheduled, or a page of memory is allocated — the kernel writes one small fixed-size record into a buffer in its own memory. The buffer holds the most recent **256** events and overwrites the oldest ones. A user program called `blackbox` reads the buffer through a new system call and prints it as a readable table.

2. The whole design on one page



3. How it works, step by step

STEP 1 Something happens in the kernel

Seven places in the kernel were given one extra line of code. That line is the only change each of those files

needed:

Where	The line added	What it means
proc.c — kfork()	eventrecord(EV_FORK, ...)	a new child process was created
proc.c — kexit()	eventrecord(EV_EXIT, ...)	a process is finishing
proc.c — scheduler()	eventrecord(EV_SCHED, ...)	a CPU picked a process to run
exec.c — kexec()	eventrecord(EV_EXEC, ...)	a process loaded a new program
syscall.c — syscall()	eventrecord(EV_SYSCALL, ...)	any system call was made
kalloc.c — kalloc()	eventcur(EV_ALLOC, ...)	a page of memory was handed out
kalloc.c — kfree()	eventcur(EV_FREE, ...)	a page of memory was returned

Why this is the small way: the syscall hook sits in the one dispatcher that every system call passes through. One line covers all 24 calls instead of editing 24 functions.

STEP 2 eventrecord() writes one record

Every event is the same shape — 40 bytes, no strings, no pointers to anywhere else:

```
struct kernel_event {
    uint64 seq;    // 0, 1, 2, 3, ... never repeats
    uint64 arg1;   // meaning depends on the type
    uint64 arg2;
    uint  ticks;   // when: timer ticks since boot
    int   pid;     // who: which process
    int   cpu;     // where: which CPU core
    int   type;    // what: FORK / EXIT / EXEC / SYSCALL / ...
};
```

Filling this in is about a dozen stores to memory. There is no loop, no string formatting and no memory allocation, because this code runs on the kernel's hottest paths.

STEP 3 The record goes into a circular buffer

The buffer is a plain fixed array of 256 records that exists for the whole life of the kernel. A variable head points at the next free slot. After each write:

```
head = (head + 1) & 255;    // 255 = 256 - 1

before:  [ 12 ][ 13 ][ 14 ][  _ ][  _ ]    head = 3
write:   [ 12 ][ 13 ][ 14 ][ 15 ][  _ ]    head = 4
full:    [ 12 ][ 13 ][ 14 ][ 15 ][ 16 ]    head = 0 (wrapped)
next:    [ 17 ][ 13 ][ 14 ][ 15 ][ 16 ]    oldest overwritten
```

Why 256: it is a power of two, so wrapping around is a bitwise AND instead of a division. Division is slow; AND is one instruction.

Why a fixed array: the recorder must never call `kalloc()`. If it did, and memory events were switched on, allocating space for an event would record an event, which would allocate space, which would record an event — infinite recursion. A fixed array cannot do that. It also means the recorder can never run out of memory or crash the kernel.

STEP 4 One lock, and it is the smallest kind

Several CPUs can hit the recorder at the same moment, so the buffer is protected by a spinlock, `elog.lock`. It follows one strict rule:

elog.lock is a leaf lock. While holding it, the recorder never takes another lock, never allocates, never prints and never

sleeps. A lock that never waits for another lock can never be half of a deadlock cycle. That single rule is what makes it safe

to call the recorder from inside the scheduler, which is already holding a process lock.

STEP 5 User space asks for the events

Two new system calls were added — numbers 23 and 24:

System call	What it does
<code>getevents(buf, max)</code>	copies up to <i>max</i> events, oldest first, into your buffer; returns how many were copied, or -1 on a bad argument or address
<code>eventctl(mask)</code>	negative = ask which event types are being recorded; zero or more = record exactly these types and empty the buffer. Returns the previous setting

STEP 6 The kernel copies out carefully

Events are copied 8 at a time onto the kernel stack, the lock is released, and only then are they written into user memory with `copyout()`.

Why not just copy while holding the lock? Writing to user memory can hit a page that is not present yet. Handling that fault calls `kalloc()`, which calls the recorder, which tries to take `elog.lock` — on the same CPU that already holds it. xv6 spinlocks are not recursive, so the kernel would panic. Staging in small chunks avoids the whole problem.

STEP 7 blackbox prints the table

The kernel stores only numbers. Turning event type 4 and argument 16 into the text `SYSCALL write` happens entirely in the user program.

Why: printing is slow and needs the console lock. Doing it in the kernel would mean the act of recording an event changed the timing of the thing being recorded. Keeping all formatting in user space keeps the kernel side down to a few stores.

4. What you actually see

Real output, captured after running `echo hi` at the shell. Read it top to bottom as a story: the shell (pid 2) reads your typing one character at a time, forks a child, the child gets scheduled on a CPU, and then execs the program.

```
$ blackbox -n 14
```

```
===== XV6 KERNEL EVENT RECORDER =====
```

SEQ	TIME	CPU	PID	EVENT	DETAILS
156	47	0	2	SYSCALL	read
157	47	0	2	SYSCALL	read
...					(six more reads)
163	47	0	2	SYSCALL	fork
164	47	0	2	FORK	child=5
165	47	0	2	SYSCALL	wait
166	47	0	5	SCHEDULE	
167	47	0	5	SYSCALL	sbrk
168	47	0	5	SYSCALL	exec
169	47	0	5	EXEC	blackbox

Total events shown: 14 (seq 156..169)

=====

Column	Meaning
SEQ	event number since boot. A jump in this column means older events were overwritten
TIME	timer ticks since boot
CPU	which core recorded it (this run had 3)
PID	which process the event is about
EVENT	one of FORK, EXIT, EXEC, SYSCALL, ALLOC, FREE, SCHEDULE

5. How to run it

Build and boot

```
cd xv6-blackbox
make qemu
```

Wait for the `$` prompt. To leave qemu at any time press **Ctrl-A**, release, then press **x**.

Look at the log

```
$ blackbox          # print everything in the buffer
$ blackbox -n 20     # print only the 20 most recent events
$ blackbox -s        # show which event types are being recorded
$ blackbox -c        # empty the buffer
```

Choose what gets recorded

```
$ blackbox -m fork,exit    # only process creation and exit
$ blackbox -m sched        # only scheduling decisions
$ blackbox -m alloc,free   # watch physical memory (off by default)
$ blackbox -m all          # everything
$ blackbox -m default      # back to normal
```

Why ALLOC and FREE are off unless you ask: the page allocator runs tens of thousands of times per second. Leaving

it on would fill all 256 slots faster than a human can type the next command, and everything else would scroll away.

A demonstration worth trying

```
$ blackbox -m fork,exit,exec  # narrow the recording, clears the buffer
$ echo hello                  # do something ordinary
$ blackbox                    # see exactly what the kernel did
```

Run the tests

```
$ eventtest          # all 8 tests
$ eventtest 6        # just test 6
```

Expected last line: `=== ALL TESTS PASSED ===`

Test	Checks
1	events are recorded; timestamps and sequence numbers are sane
2, 3	fork and exit are recorded, with the right child pid and exit status
4	system calls are recorded — and getevents never records itself
5	scheduling events appear for more than one process
6	writing 512 events into a 256-slot buffer overwrites cleanly and does not crash
7	CPU numbers are valid with several cores running at once
8	an empty log, a zero count, a negative count and a bad pointer are all handled

6. Every file that was touched

File	Lines	Why
kernel/event.h	new	the record layout and event type constants, shared by both sides
kernel/event.c	new	the ring buffer, the lock, and the two system call bodies
user/blackbox.c	new	the viewer that formats and prints the log
user/eventtest.c	new	the 8-test suite
kernel/proc.c	3 lines	hooks in fork, exit and the scheduler
kernel/kalloc.c	2 lines	hooks in the page allocator
kernel/exec.c	1 line	hook in exec
kernel/syscall.c	1 line	one hook covering every system call
kernel/main.c	1 line	eventinit() at boot, after kinit()
kernel/sysproc.c	—	argument checking for the two new system calls
defs.h, syscall.h, user.h, usys.pl, Makefile	—	declaring and wiring up the new calls

7. The six ideas to remember

Decision	The reason in one line
Fixed 256-slot array, never grows	the recorder can never allocate, so it can never recurse into itself or run out of memory
Size is a power of two	wrapping is one AND instruction instead of a division
elog.lock is a leaf lock	a lock that never waits for another lock cannot deadlock
Never copy to user memory while holding the lock	a page fault during the copy would re-enter the recorder and panic
All formatting in user space	keeps the kernel side to a few stores, so recording barely disturbs what it records
getevents and eventctl are not traced	otherwise every read of the log would first write to it